SIH26037 — Round 2 ML Evaluation & Verification Plan

Role: Kishan (ML — Independent Evaluation & Verification) | **Stream:** ML Track

Context: Smart India Hackathon 2026 | MathWorks Problem Statement SIH26037

Timeline: 2 Days Demo: Saturday 12:30pm Scope: S1 & S2 (No new scenarios)

Independent Check: Never trust on word

Core Persona & Operational Directives

Primary Function: You do not grade your own work, and you do not let Shourya grade his. Every number reported for the final demo and presentation must be independently verified by re-running inference. If a number cannot be computed, quote `TODO(unverified)` — never invent or round up.

Non-Negotiable Guardrails:

- **Section 3 of AGENTS.md is frozen:** Never alter the S1–S10 perception/planner contracts.
- **Never touch matlab/baseline/:** That is the control baseline. Editing it invalidates all findings.
- **No Single-Number Summaries:** Report 12-fold metrics strictly as `mean ± std`.

Master Task Execution Schedule

ID	Task Name	Schedule	Target Artifact / File	Key Metrics / Outputs	Verification & Stop Rule
Task 0	Calibration Logic Sync	Day 1 Early AM	`ml/python/model/calibrate.py`	Fitting logic handoff from Shourya	Must obtain fitting code by midday Day 1 to embed into CV folds.
Task 1	Verify Shipping Yield Models	Day 1 Morning	`ml/python/model/evaluate.py`	Dangerous error rate, ECE, reliability curves	**STOP RULE:** Discrepancy $\to$ stop and notify Shourya & Aditya immediately.
Task 3	Build Comparison Baseline	Day 1 PM	`ml/python/model/baseline.py`	Logistic Regression AUC-ROC, Brier, ECE	Trained on compact 5-feature subset under identical 12-fold protocol.
Task 2	12-Fold LOSO Evaluation	Day 1 PM – Day 2 AM	`ml/python/model/cross_validate.py`	`mean ± std` across 12 folds, pooled reliability curve	1 seed/fold. Call out worst-performing station by name with qualitative root cause.

ID	Task Name	Schedule	Target Artifact / File	Key Metrics / Outputs	Verification & Stop Rule
Task 4	Paired Significance Test	Day 2 Morning	`ml/python/model/cross_validate.py`	Paired $\Delta \pm \text{std}$, Wilcoxon p-value	Non-parametric signed-rank test. Honest reporting of win or null result.
Task 5	Verify YOLOX Domain Gap	Day 2 Midday	MATLAB detector evaluation	Before/after dangerous error rate vs $\le 1\%$	Verify actual exported `.mat` detector on raw test frames.

Detailed Task Specifications

Task 1 — Independently Verify Shourya's Shipping Models (Fast Check)

- **Input:** Calibrated PyTorch checkpoints (`yield_lstm` and `yield_attention`) from Shourya.
- **Protocol:** Re-run evaluation on the frozen validation set via `evaluate.py` .
- **Verification Check:** Recompute the dangerous error rate ($FP / (TP + FP)$ at the operating threshold) and ECE.
- **Trap to Prevent:** Confirm that the reported "uncalibrated" baseline was not accidentally evaluated on already-scaled logits.
- **Action on Discrepancy:** Halt immediately and report to Shourya and Aditya. Do not silently resolve discrepancies.

Task 2 — 12-Fold Leave-One-Station-Out (LOSO) Cross-Validation

- **Why:** Standard random train/test splits leak station-specific camera artifacts. Splitting across the 12 distinct geographic stations proves genuine out-of-distribution generalization.
- **Implementation (`cross_validate.py`):**
  1. Iterate through stations $k \in \{1 \dots 12\}$.
  2. Train model on the remaining 11 stations.
  3. Reserve an internal sub-split of the 11 stations for validation/early stopping (never touch station k).
  4. Fit calibration logic (Platt scaling / temperature scaling) within the fold.
  5. Evaluate on station k: AUC-ROC, Brier score, ECE, F1 at asymmetric threshold, sample count.
- **Reporting:** Aggregate all metrics as `mean ± std` . Generate one unified pooled reliability diagram combining all held-out predictions. Identify the worst-performing station with an environmental explanation (e.g. night illumination, dense occlusions).

Task 3 — Build the Comparison Baseline (Logistic Regression)

- **Why:** In literature, high-capacity neural networks often fail to outperform a well-regularized linear baseline on unstructured trajectory data. We must prove our neural architecture earns its keep.
- **Implementation (`baseline.py`):** Train a `LogisticRegression` model using a hand-picked feature subset:
  - `tau` (Feature 10: time-to-contact $h / (dh/dt)$)
  - Lateral closure rate (Feature 11)
  - Ego vehicle speed (Feature 28)
  - Candidate ego action (Feature 31)
  - Coarse 3-way actor class (Vehicle / VRU / Animal collapsed from 16-way one-hot)
- **Protocol:** Run across the exact same 12 folds as Task 2 for an apples-to-apples comparison.

Task 4 — Paired Statistical Significance Testing

- **Protocol:** Calculate paired differences $\Delta_i = \text{Metric}_{\text{LSTM}, i} - \text{Metric}_{\text{Baseline}, i}$ across the 12 matched folds.
- **Method:** Execute `scipy.stats.wilcoxon(diffs)` . Non-parametric testing is mandatory given the 12-sample size.
- **Outcome:**
  - *Significant ($p < 0.05$):* Claim defensible statistical superiority of the recurrent architecture.

- *Not Significant ($p \ge 0.05$):* Report as an honest engineering finding showing current complexity boundaries.

Task 5 — Independently Verify Shourya's YOLOX Domain-Gap Fix

- **Objective:** Verify whether fine-tuned YOLOX reduces the dangerous detection error rate from 20.18% down to the $\le 1\%$ threshold required for safety gating.
- **Execution:** Load Shourya's exported MATLAB detector artifact and test frames. Re-run inference and tabulate false negatives on vulnerable road users (cows, auto-rickshaws, pedestrians).

Team Handoff Matrix

Recipient	Deliverables	Impact on Demo / Presentation
Aditya (Lead)	12-Fold Table (mean ± std), Pooled Reliability Plot, Wilcoxon p-value, Worst-Station Analysis	Core empirical evidence cited in the technical pitch and jury Q&A.
Aditya B. (Bridge)	Verified operating metrics, calibration status, and gating conditions	Populates live Model Status HUD panel in `sc.plannerView` with verified disclosures.
Shourya (ML Train)	Immediate discrepancy reports on shipping checkpoints and YOLOX	Prevents invalid checkpoints or uncalibrated models from propagating to MATLAB.

> **Standby Notice:** This plan is locked and ready for execution. No source code modifications or model runs will occur until the operator explicitly issues the single-word prompt `start`.